

Codebook of UltraGrammer

ahbiee

Contents

1 你是否有...

1.1 嘗試...

- **再讀一遍題目**
- 懷疑你認為「理所當然」的直覺?
- 試著寫下直覺的規律? (有助於觀察)
- 換個方向思考?
- 暴力解?
- 貪心解?
- 動態規劃 (DP)?
- 狀態建模, 把問題轉化成圖論?
- 只在奇數 index 反轉狀態?
- 用乘法取代除法? ($/2 = *2^{-1}$, 模逆元)

1.2 檢查...

- **確定沒漏看敘述?**
- 確認數值範圍? (溢位, 精度位數, 陣列大小...)
- 不同的陣列/字串使用同個 index 導致超出範圍?
- 模板抄錯?
- 多筆資料之間初始化?
- 題目額外的敘述, 暗示?
- 未定義行為? (陣列越界, 未初始化)
- overflow? (int $\rightarrow$ long long)
- 非 void function 忘記 return?
- 浮點數誤差? (極小的 eps, 是否需要 long double)
- 隱轉換? (int*int 爆了才存到 long long)
- signed 與 unsigned int 之間的 compare?

2 資料結構: 前綴和

2.1 BIT (Fenwick Tree)

用法: 支援單點修改與區間求和。index 是 1-based。
時間複雜度: $\mathcal{O}(\log n)$

<MINTED>

2.2 Segment Tree 線段樹

用法: 給定 n 筆資料查詢與 m 次範圍更新, 解決多次範圍更新的問題, 找區間和或最大/小值

時間複雜度: 建樹 $\mathcal{O}(N)$ 查詢/更新 $\mathcal{O}(\log N)$

<MINTED>

3 數論: 尤拉公式, 數論分塊, Sprague-Grundy 函數, 排容原理

3.1 快速冪

用法: 計算 $a^b \pmod{m}$ 。

時間複雜度: $\mathcal{O}(\log b)$

<MINTED>

3.2 模逆元與費馬小定理

用法: 定義: 若 p 為質數, 且整數 a 與 p 互質, 則 $a^{p-1} \equiv 1 \pmod{p}$ 。

邏輯應用: 兩邊同除以 a 可得 $a^{p-2} \equiv a^{-1} \pmod{p}$ 。

這代表計算 $(x/a) \pmod{p}$ 等同於計算 $(x \cdot a^{p-2}) \pmod{p}$ 。

時間複雜度: $\mathcal{O}(\log p)$

<MINTED>

3.3 質數檢查

用法: 單次詢問大數用 isPrime; 若有大量詢問則建質數表。

時間複雜度: $\mathcal{O}(\sqrt{N})$

<MINTED>

3.4 大數乘法

用法: 未知長度的大數相乘

時間複雜度: $\mathcal{O}(N \times M)$

<MINTED>

4 字串

4.1 KMP 字串匹配

用法: 尋找 pattern 出現位置。pi 陣列可求最小循環節。

時間複雜度: $\mathcal{O}(N + M)$

<MINTED>

4.2 Z-Algorithm 字串匹配

用法: z[i] 代表 S 與 $S[i \dots N-1]$ 的最長共同前綴。

時間複雜度: $\mathcal{O}(N)$

<MINTED>

4.3 Manacher 迴文判斷

用法: 求最長迴文子字串。預處理字串塞入 # 將所有迴文轉為奇數長度。

時間複雜度: $\mathcal{O}(N)$

<MINTED>

4.4 Trie 字典樹

用法: 字串前綴統計或最大 XOR 問題。

時間複雜度: $\mathcal{O}(|S|)$

<MINTED>

4.5 AC 自動機

用法: 多字串匹配 (Trie + KMP)

時間複雜度: $\mathcal{O}(N + M)$

<MINTED>

5 圖論: Bellman-Ford, Floyd, SPFA 最短路徑, 著色問題, 帶權二分圖最大匹配 (KM 演算法), 無權二分圖最大匹配 (匈牙利演算法), 最大流

需注意:

是否為全連通圖 (connected, 只有一個 component), 對於連通圖要找 region 可以用 Euler's theorem $V-E+R=2$

undirected 無自環

directed

weighted 有權重 (點或邊)

multigraph 兩點之間有多條直接的路徑

有向圖的 outdegree, indegree, 無向圖的 degree

表示圖的方式: Adjacency matrix, adjacency list

5.1 BFS 與拓撲排序

用法: 無權圖最短路徑 / 拓撲排序計算 In-degree 尋找相關順序, 可判斷是否有環。

時間複雜度: $\mathcal{O}(V + E)$

<MINTED>

5.2 DFS

用法: 用遞迴走訪。

時間複雜度: $\mathcal{O}(V + E)$

<MINTED>

5.3 Dijkstra 最短路徑

用法: 單源最短路徑, 不支援負權邊, 支援自環。

時間複雜度: $\mathcal{O}((V + E) \log V)$

<MINTED>

6 幾何：三點求圓，向量，凹凸多邊形面積，點與線等等的關係，凸包

7 樹論

7.1 Disjoint Set

用法：解決集合、快速合併的問題。優化必須路徑壓縮，可選小掛大
時間複雜度： $\mathcal{O}(1)$

<MINTED>

7.2 Kruskal 最小生成樹

用法：需搭配並查集使用。將所有邊排序後，利用併查集檢查是否形成環。

時間複雜度： $\mathcal{O}(E \log E)$

<MINTED>

8 DP: LIS, LDS, 分錢幣問題, Kadane's algo 找最大子序列問題

DP 通用解題四步驟：

1. 定義陣列意義：明確寫出 $dp[i]$ (或 $dp[i][j]$) 代表什麼。
2. 找出遞迴關係式：推導出當前狀態如何由過去狀態轉移而來。
3. 設定邊界條件：初始化起點或基礎 dp 值 (如： $dp[0] = 0$)。
4. 確認遍歷順序：決定迴圈是從前到後 (Bottom-Up) 還是後到前 (Top-Down) 計算。(通常是 Bottom-Up，思考”我是如何到這一步的”去推)

8.1 01 Knapsack 背包

用法：解決物品不可拆分的情況，若可拆分用 Greedy 拿比例最佳的部分
時間複雜度： $\mathcal{O}(N)$

<MINTED>

8.2 LCS 最長共同子序列 (Sequence)

用法：二維 DP 與 Backtrack 找原始子序列

時間複雜度： $\mathcal{O}(NM)$

<MINTED>

8.3 跳格子

用法：給定一組陣列，固定跳躍距離，問你最少需要跳幾次，以及所跳過的格子 index

時間複雜度： $\mathcal{O}(N \times K)$

<MINTED>

9 Greedy: 二分搜

擴充一：區間合併 (Merge Intervals) 情境：給你一堆區間，把所有重疊的部分合併起來 (Leetcode 56)。貪心策略：依照「開始時間」排序。如果下一個區間的開始時間 $\leq$ 當前區間的結束時間，則合併 (更新當前結束時間為兩者最大值)；否則將當前區間放入答案，換下一個。擴充二：多點排程 / 優先佇列貪心 (Huffman 概念) 情境：經典的「切木板問題」或「合併果子 (碎紙機)」，每次合併兩堆，成本是兩堆的重量和，求最小總成本。貪心策略：這裡不需要排序和掃描線，而是直接把所有元素丟進 Min-Heap (Priority Queue)。每次 pop 出最小的兩個相加，把總和加進成本，再把總和 push 回 Heap，直到 Heap 只剩下一個元素。

9.1 一維區間全覆蓋

用法：需要先把輸入轉化成一個包含 l, r 的 struct，自訂 operator<，起始位置由小到大排序

時間複雜度： $\mathcal{O}(N \log N)$

<MINTED>

9.2 間格跳躍

用法：給定一組陣列，距離為 index 的值，問你最少需要跳多少次才能到終點

時間複雜度： $\mathcal{O}(N)$

<MINTED>

9.3 最大不重疊區間

用法：對一組任務做排程，請問最多可以移除多少個任務 -> 最早結束的先選。

時間複雜度： $\mathcal{O}(N \log N)$

<MINTED>

9.4 最小覆蓋區間

用法：給定一組陣列，包含開始時間與結束時間，求最少需要幾個房間就能完成所有任務

時間複雜度： $\mathcal{O}(N \log N)$

<MINTED>

10 其他: Random Select(QuickSort), 雙指針, 滑動窗口, C++ algorithm 庫

10.1 模板

用法：競賽用基礎模板

時間複雜度： $\mathcal{O}(\text{HowFastYouType})$

<MINTED>